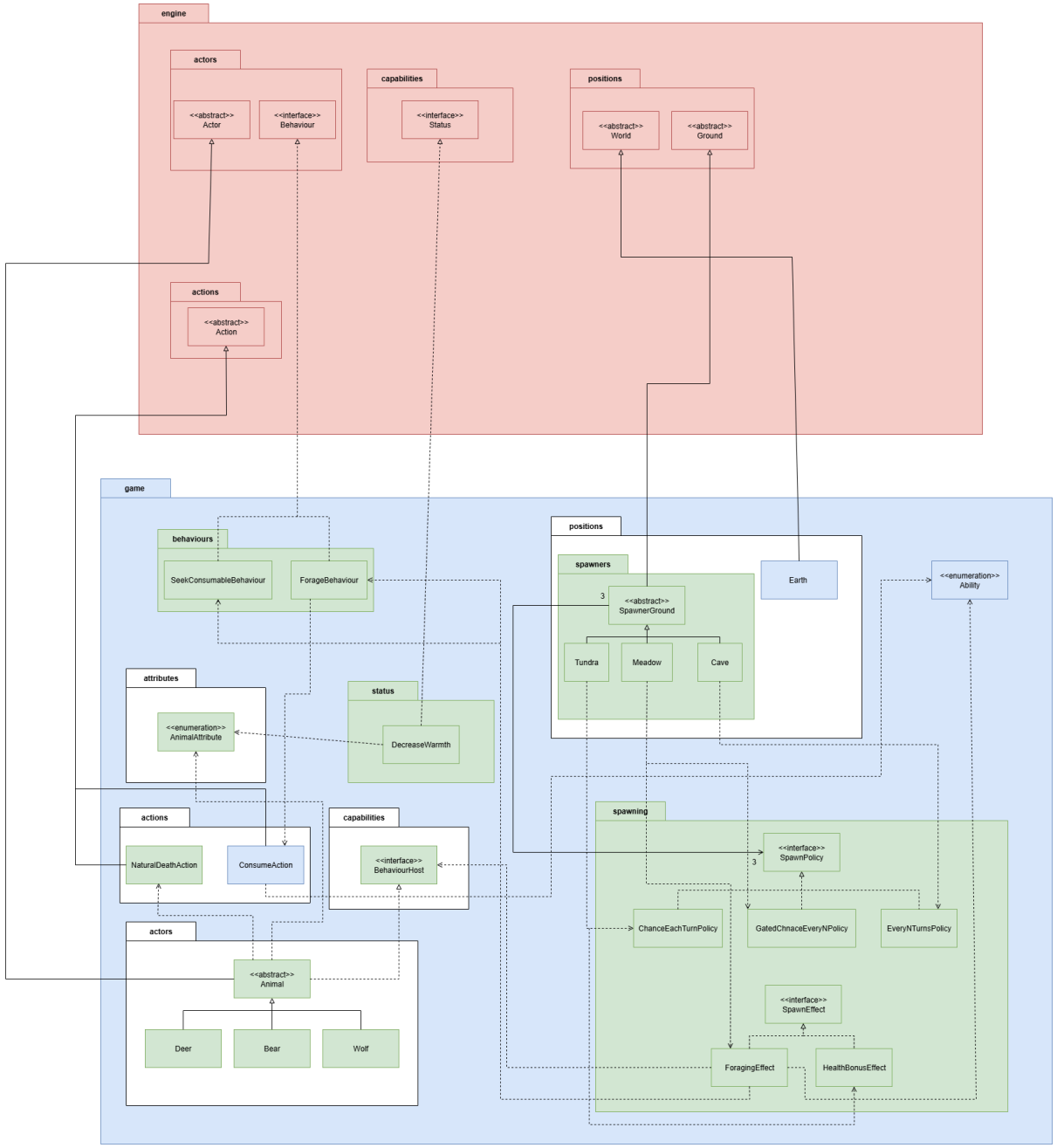
FIT 2099 Assignment 2 Design Documentation

Design Diagram

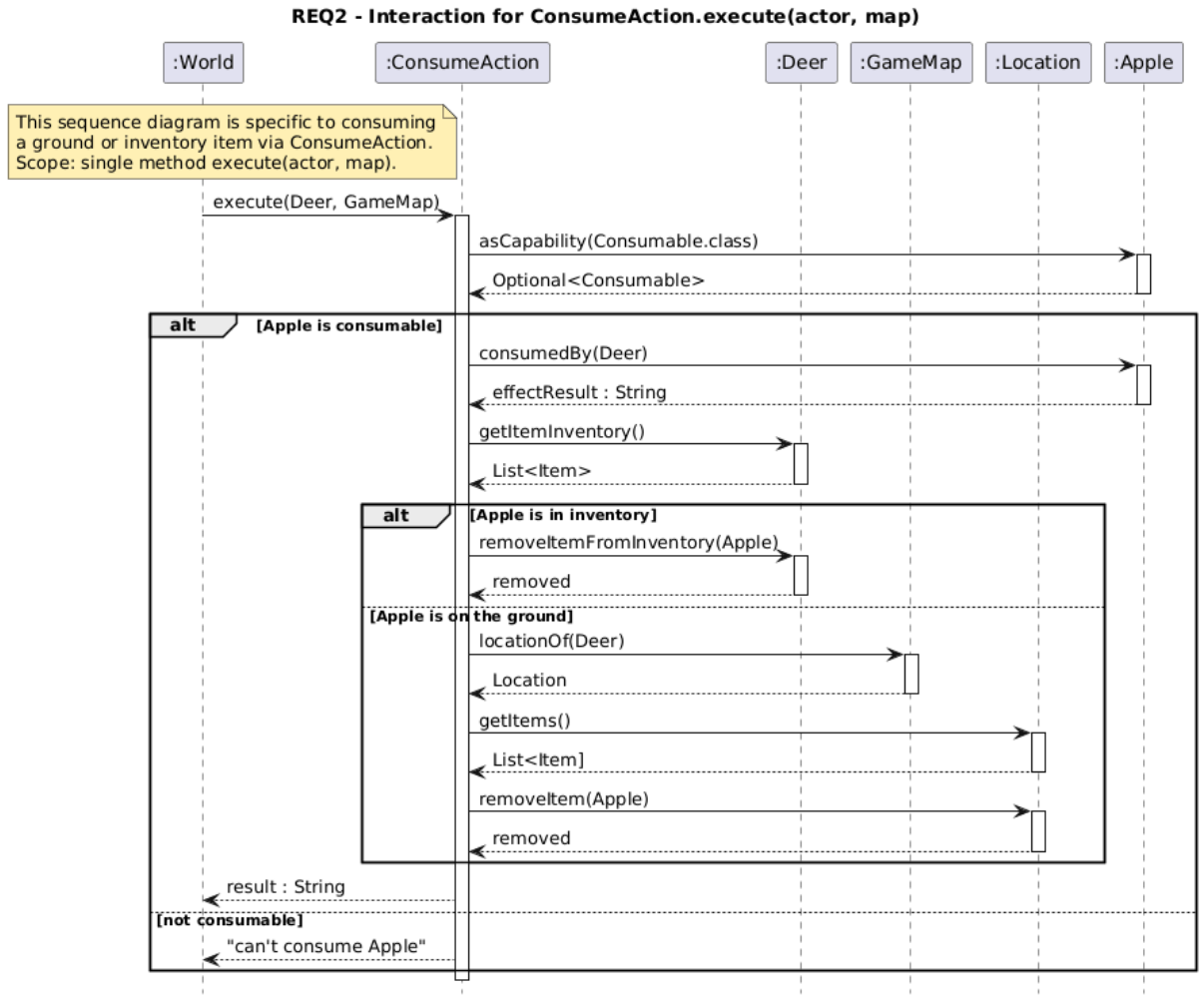
REQ2 UML Class Diagram:



Red: existing classes
Blue: old revised classes
Green: new classes

REQ 2 Sequence Diagram:

`ConsumeAction.execute(actor, map)` — it shows how a meadow-spawned animal (or player) eats ground/inventory items via the shared action/capability flow, without becoming a full “system interaction”.



Design Rational:
REQ2

Design 1: hard-codes spawning and eating logic directly into terrain and item classes.

1. Spawner logic inside each Ground: `Tundra`, `Cave`, `Meadow` each implement their own tick counters, RNG, spawn rates, and species selection inline. Equal chance is handled with local `if/else` or switch blocks.

2. Food detection with `instanceof`: Animals check ground items using `instanceof Apple/Hazelnut/YewBerry` and branch accordingly.

3. `Items remove themselves: Each consumable's consumedBy applies effects and removes itself from inventory/ground.`

4. Greedy movement toward food: Animals move to whichever adjacent tile reduces Manhattan distance to the nearest food; no pathfinding queue.

This works for a small feature set, but it couples timing, spawning, and item semantics tightly to specific classes, making change risky and repetition inevitable.

Pros	Cons
Simple to read; "all in one place" per terrain; fewer indirections for a newcomer.	Duplicates timing/random logic across terrains; any new rule requires editing many files; violates SRP/DRY and OCP; higher regression risk.
Trivial to code via local `if/else` or `switch`.	Hard to extend (must edit code to add species); testability suffers; no unified place to vary probabilities.
Straightforward; easy to understand.	Brittle: every new consumable requires code edits; encourages long `if/else` chains; violates OCP; increases coupling.Brittle: every new consumable requires code edits; encourages long `if/else` chains; violates OCP; increases coupling.
"Self-contained" feeling in item code.	Breaks symmetry ground vs. inventory; easy to forget one path , hence duplicate removals or missed removals; harder to unit test consistently.
Minimal code; performs OK in open maps.	Fails behind obstacles; can oscillate; no guarantee of reaching food; degrades UX in real maps.
Very low mental overhead initially.	Becomes expensive quickly: adding

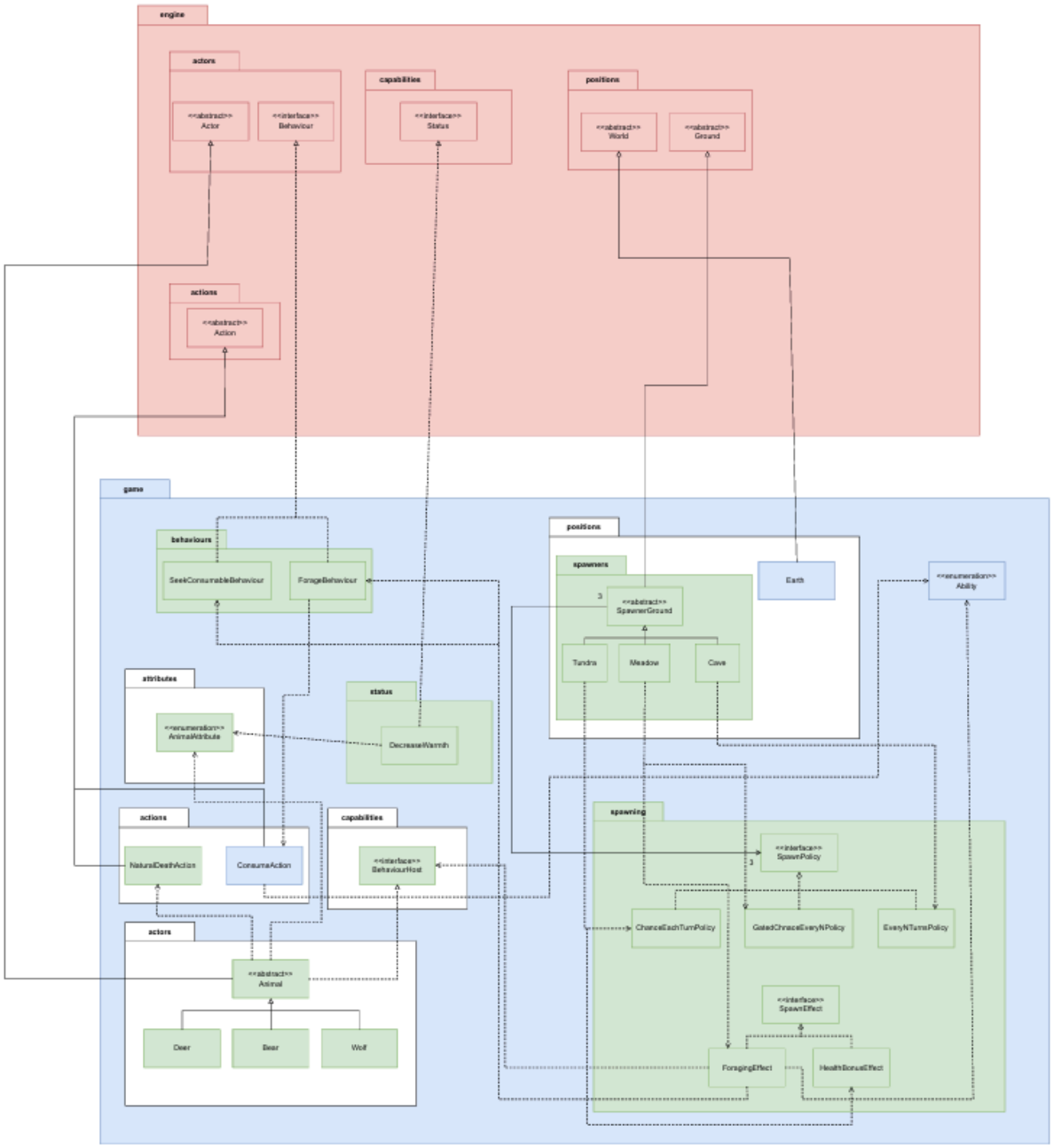
	spawners/animals/consumables multiplies edits; higher chance of bugs and inconsistencies.
Low initial complexity.	Technical debt grows fast; scattered logic makes refactors risky and time-consuming.
Few abstractions to mock.	Hard to isolate behaviours; many code paths hidden inside big methods; flaky tests due to RNG embedded in terrains.

Design 2: decomposes animal spawning and foraging into small, composable parts:

1. SpawnerGround coordinates spawning with policies (timing/probability) and effects (newborn mutations). Factories (`Supplier<Actor>`) give equal chance across species.
2. Policies (ChanceEveryTurn, EveryNTurns, GatedChanceEveryN) cleanly encode tundra/cave/meadow rules without duplication.
3. Effects add terrain-specific behaviour: tundra health buff; meadow Forage (-1) and Seek (0, BFS) so animals immediately eat or move toward food.
4. Animals expose BehaviourHost (capability, not `instanceof`), keep a priority-ordered behaviour map, and use Warmth + NaturalDeathAction for engine-aligned removal.
5. Consumables implement a Consumable capability; ConsumeAction centrally removes items from inventory/ground after applying effects—so player and animals share the same consumption semantics.

Pros	Cons
Strong SRP/OCP; zero duplication; easy to add new terrains/rules or species by composition; clean unit testing (mock policies/effects).	Slightly more indirection than hard-coded logic; newcomers need to learn the pattern.
Equal probability “for free”; plug-and-play new species; no switch/if chains.	Requires discipline to register factories at map setup.
Clear timing/probability; deterministic cave spawning; tundra 5% is a one-liner.	More classes to navigate (small abstractions).
Reusable (+10 HP tundra buff; meadow foraging/seek); can stack multiple effects safely.	Order of multiple effects should be considered if side effects ever depend on one another.
Intentional priority; animals eat immediately or path reliably (BFS) toward food; combat/wander are clean fallbacks.	Priority tuning is a design choice—teams must keep a shared convention.

No `instanceof`; high extensibility; items/actors remain decoupled; DIP-compliant.	Requires implementing `asCapability` consistently on new types.
One place removes items (ground or inventory); consistent effects for player/animals; easy to test.	Items must not self-remove; contributors need to follow the pattern.
No engine changes; uses standard `unconscious(map)` path; exceptions (spawn collisions) safely handled.	Spawn failures are skipped for that tick; optional logging recommended if debugging complex maps.
Scales well as features grow; low regression risk; clear unit boundaries.	Slight upfront design cost compared to “just code it in the ground class”.



The diagram above shows the final design of requirement 2, which utilise Design 2 stated above. Automatic spawning is delegated to configurable spawners; terrain rules are expressed as policies; newborn bonuses/behaviours are attached as effects; animals act through a priority-based behaviour map; and consumables are integrated via capabilities so animals and players share identical item effects.

Spawner infrastructure.
`SpawnerGround` (abstract ground) composes three orthogonal parts:

- `SpawnPolicy` (when to spawn):
  `ChanceEveryTurnPolicy(0.05)` for Tundra; `EveryNTurnsPolicy(5)` for Cave; `GatedChanceEveryNPolicy(7, 0.5)` for Meadow.

- Factories (`Supplier<Actor>`) (what to spawn): equal probability across registered species.

- `SpawnEffect` (how to modify newborns):
  `TundraEffect` (+10 HEALTH, optional cold resistance).
  `ForagingEffect` (Meadow) attaches `ForageBehaviour` at −1 (eat on tile) and `SeekConsumablesBehaviour` at 0 (BFS one step toward the nearest *reachable* consumable; radius tunable).

Tick flow: `policy.onTick()` → skip if occupied/empty → if `policy.shouldSpawn()` → pick factory uniformly → create actor → apply effects → `location.addActor(newborn)` (placement exceptions are caught and the tick is skipped safely).

Animals & behaviours.
`Animal` exposes `BehaviourHost` via `asCapability`, holds a `TreeMap<Integer, Behaviour>` (deterministic priority), adds WARMTH with `DecreaseWarmth`, and returns `NaturalDeathAction` if unconscious so the engine removes the actor and prints the standard message. Species (`Bear`, `Wolf`, `Deer`) keep constructors minimal (weapons + Hostile/Wander); spawner effects layer food behaviours without editing species.

Consumables & consumption.
`Apple`, `Hazelnut`, `YewBerry` implement `Consumable`. Their `consumedBy` applies effects only; a single `ConsumeAction` removes the item from inventory or ground after the effect. Meadow-spawned animals print a friendly forage message (tagged by a marker ability the effect sets).

Map wiring (spec compliance).
Forest: ≥1 Tundra (Bears), ≥1 Cave (Bear/Wolf/Deer), ≥1 Meadow (Deer).
Plains: ≥1 Tundra (Wolves), ≥1 Cave (Bear/Wolf), ≥1 Meadow (Deer/Bear).

Single Responsibility (SRP).

- `SpawnerGround` orchestrates spawning, policies time it, effects mutate newborns, behaviours act.

Open–Closed (OCP).

- New terrain rule = new policy/effect; new species = new factory; existing classes remain closed for modification.
  Liskov / ISP / DIP.

- Spawners are grounds; animals are actors; items expose a Consumable capability. We depend on abstractions (policy/effect/capability), not concrete classes.
  DRY & KISS.

- One `ConsumeAction` centralises removal; explicit priorities (−1 forage → 0 seek → 1 hostile → 999 wander); the BFS is tiny yet robust.

Justification for changes from Assignment 1 (A2-relevant)

1. Manual placement → automatic spawners.
   A1's ad-hoc placement did not scale. We introduced `SpawnerGround` with policies and factories to remove duplication and enable per-tile configuration (meets REQ2's "several spawners" requirement).

2. Inline logic → composition (policy/effect).
   Terrain-specific branching in A1 would have grown rapidly. Separating timing (policy) from mutation (effect) keeps classes small and extensible (aligns with OCP/DIP).

3. Item handling and foraging.
   A1 relied on player-centric consumption. We introduced the `Consumable` capability and `ConsumeAction` so animals and player share effects; meadow newborns gain foraging/seek behaviours via effect (no species edits).

4. Warmth & engine-aligned death.
   We added WARMTH and `DecreaseWarmth`, and we route KO through the engine's `unconscious` path (`NaturalDeathAction`) to keep removal consistent and testable.

Justification for the new REQ2 features

- Tundra 5% + +10 HP: Implemented by `ChanceEveryTurnPolicy(0.05)` and `TundraEffect`. The effect localises the "cold buff" to tundra-born animals, enabling different tundras to apply different effects (as the spec anticipates).

- Cave every 5 turns: `EveryNTurnsPolicy(5)` makes the cadence explicit and reusable; equal species chance comes from uniform factory choice.

- Meadow 50% every 7 turns + foraging: `GatedChanceEveryNPolicy(7, 0.5)` encodes timing, while `ForagingEffect` attaches Forage(−1) and Seek(0 BFS) so animals immediately eat or move toward edible drops. Because items expose `Consumable`, new foods automatically participate.

- Extensibility across maps/spawners: Different instances register different factory sets and effects; the same animal can be born with different bonuses/behaviours depending on where it spawned, as required.

Conclusion.
By separating when we spawn (policy), what we spawn (factory), how newborns differ (effects), and what they do each turn (behaviours with explicit priorities), the final design delivers REQ2 faithfully, integrates cleanly with the engine, and demonstrates the maintainability and extensibility